

```

}

function findSubstringRecursive(needle, haystack, i) {
    if (i === undefined) {
        i = 0;
    }

    if (i >= haystack.length) {
        return -1; // # BASE CASE (Needle not found.)
    }

    if (haystack.substring(i, i + needle.length) == needle) {
        return i; // # BASE CASE (Needle found.)
    } else {
        // RECURSIVE CASE
        return findSubstringRecursive(needle, haystack, i + 1);
    }
}

document.write(findSubstringIterative("cat", "My cat Zophie") + "<br />");
document.write(findSubstringRecursive("cat", "My cat Zophie") + "<br />");
</script>

```

These programs make a call to `findSubstringIterative()` and `findSubstringRecursive()`, which return 3 because that is the index where `cat` is found in `My cat Zophie`:

```

3
3

```

The programs in this section demonstrate that it is always possible to turn any loop into an equivalent recursive function. While replacing a loop with recursion is possible, I advise against it. This is doing recursion for recursion's sake, and since recursion is often harder to understand than iterative code, code readability deteriorates.